**CURSO: CC451 INTERACCIÓN HUMANO-COMPUTADOR**

## PRÁCTICA CALIFICADA 03: REPORTE DE AVANCE DEL PROYECTO (70%)

# Sócrates Bot

Asistente socrático con interfaz de voz y mascota virtual para aprender Python

| | |
|---|---|
| **Grupo** | Grupo 1 |
| **Integrantes** | 1. Luis Alanya Campos<br>2. Jharvy Cadillo Tarazona<br>3. Pedro Vargas Alfaro |
| **Plataforma y entorno** | Aplicación Web Progresiva (PWA), compatible con laptop y celular |
| **Capa de inteligencia artificial** | API de Groq (modelo Llama 3.3 70B) con respaldo automático local |
| **Repositorio** | github.com/Jharvichu/Socrates-Bot |
| **Nivel de avance reportado** | 70% funcional: código modular, 6 pantallas operativas, pruebas con evidencia y trazabilidad completa |

# 1. Introducción y resumen del proyecto al 70%

**Idea central:** Sócrates Bot es una aplicación web pensada para ayudar a cualquier persona que esté dando sus primeros pasos en la programación con Python. Uno de los mayores problemas al empezar a programar es que, al encontrarse con un error de sintaxis o de lógica, el estudiante no sabe qué hacer y se desmotiva rápidamente. Además, la mayoría de las herramientas de inteligencia artificial actuales entregan el código ya resuelto, lo que impide que el estudiante desarrolle su propia capacidad de razonamiento.

**El método socrático:** nuestra aplicación cambia ese enfoque. En lugar de regalar la respuesta o escribir el código, hace preguntas orientadoras y ofrece pistas pequeñas para que el propio estudiante descubra dónde está la falla y cómo solucionarla. Todo esto se acompaña de una mascota virtual, un pingüino que crece y evoluciona junto al estudiante a medida que supera retos y comprende conceptos.

**Accesibilidad real para todos:** el sistema se diseñó desde la raíz pensando en la accesibilidad universal. Puede usarlo una persona sin ninguna dificultad, una persona con discapacidad visual (que necesita que la aplicación le hable y funcione con lectores de pantalla) o una persona con discapacidad motora en las manos (que necesita controlar toda la aplicación y dictar código únicamente con su voz, sin teclado ni ratón).

**Alcance del 70%:** siguiendo lo solicitado en la guía de la Práctica Calificada 03, esta entrega corresponde a un avance del 70% del proyecto. Los componentes principales están programados, conectados y verificados con evidencia (sección 5), mientras que algunos aspectos avanzados quedan delimitados para la entrega final (sección 7).

# 2. Arquitectura y flujo de la solución

## 2.1. Arquitectura de referencia (modelo C4)

La Figura 1 muestra la arquitectura de contenedores definida en la etapa de diseño (PC02). Sirve como arquitectura objetivo del proyecto completo; la Tabla 1 indica qué tecnología se usó realmente en este avance del 70% para cada contenedor.

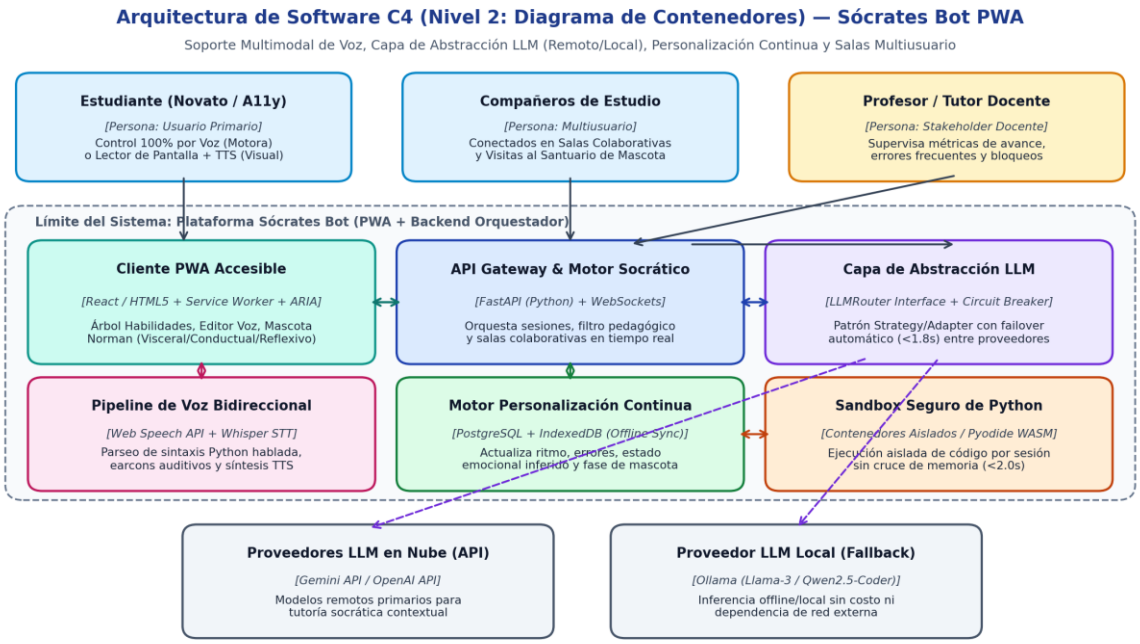



*Figura 1: Arquitectura de referencia C4 (nivel 2: contenedores) definida en la PC02.*

*Tabla 1: Correspondencia entre la arquitectura de referencia y la implementación actual.*

| Contenedor C4 | Implementación al 70% | Estado |
|---|---|---|
| Cliente PWA accesible | HTML5 semántico y JavaScript sin framework (`app.js`, `editor.js`), `manifest.json`, service worker `sw.js`, roles ARIA y región `aria-live`. | Implementado (sin React) |
| API Gateway y motor socrático | Servidor FastAPI en `main.py` con endpoints REST y WebSocket `/ws/room/{id}`. | Implementado |
| Capa de abstracción LLM | `app/socratic_tutor.py`: conector con Groq (Llama 3.3 70B) y conmutación automática a un motor socrático local basado en reglas. | Implementado (Groq en lugar de Gemini/OpenAI) |
| Pipeline de voz bidireccional | `static/js/voice_interface.js`: Web Speech API para reconocimiento y síntesis, intérprete de intenciones, dictado de Python y earcons. | Implementado con Web Speech API (sin Whisper) |
| Motor de personalización continua | `app/user_profile.py` con persistencia en `data/user_profiles.json`. | Parcial (JSON en lugar de base de datos) |
| Sandbox seguro de Python | `app/code_sandbox.py`: subproceso independiente con límite de 2.0 s y lista de operaciones bloqueadas. | Parcial (Docker para el 100%) |
| Proveedor LLM local | Motor de reglas pedagógicas local usado como respaldo. | Parcial (Ollama para el 100%) |

### 2.2. Flujo entre pantallas

La aplicación se organiza en seis pantallas (P1 a P6). La Figura 2 resume las transiciones entre ellas; cada transición tiene un comando de voz equivalente y se anuncia mediante `aria-live` para los lectores de pantalla.

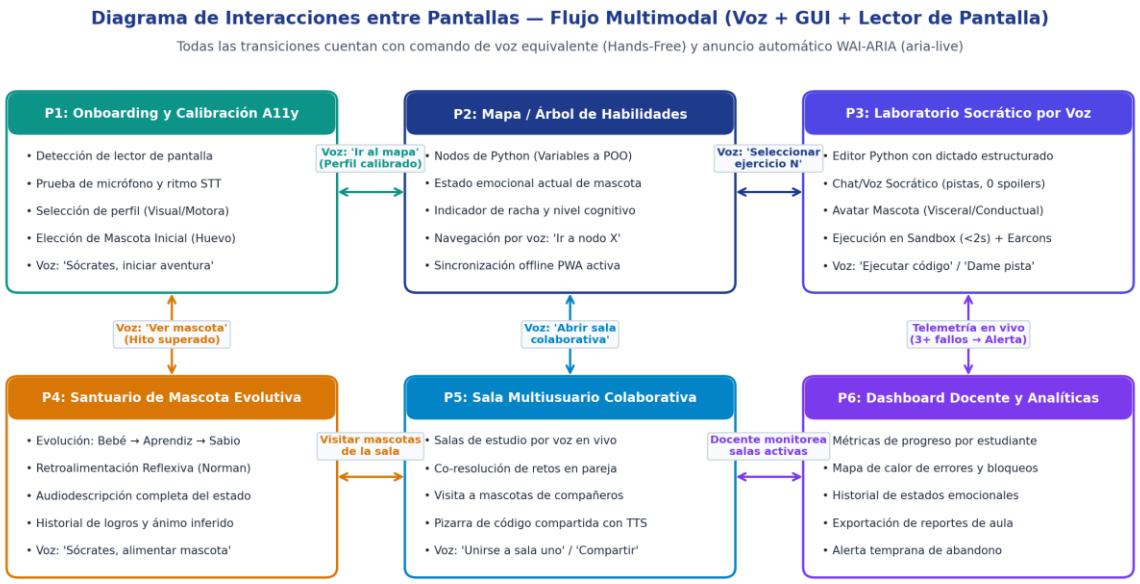



*Figura 2: Diagrama de interacciones entre pantallas (flujo multimodal).*

# 3. Cumplimiento de los requerimientos obligatorios de la PC03

### 3.1. Interfaz de voz (reconocimiento y síntesis)

Para que la aplicación pueda usarse con manos libres y resulte cómoda para personas con problemas de movilidad o visión, se implementó un sistema de voz bidireccional con la Web Speech API del navegador:

- **Lenguaje natural, no frases exactas:** antes de interpretar una orden, el sistema normaliza la transcripción (quita tildes y signos, convierte números dichos con palabras como «quince» en 15) y la compara con una tabla de intenciones con sinónimos. Así, «Ejecutar código», «corre el programa» o «pruébalo» producen la misma acción.
- **Uso de las alternativas del reconocedor:** el navegador entrega hasta tres transcripciones posibles por frase. El sistema prueba cada una y, en el dictado, elige la que forma una instrucción de Python válida (por ejemplo, prefiere «si edad es mayor a 18» sobre «ciudad es mayor a 18»).
- **Dictado de código Python:** convierte frases como «escribe si edad mayor que dieciocho» en `if edad > 18:`, inserta el código en la posición del cursor y maneja la sangría automáticamente (bloques `if`/`else`, `for`, `while` y `def`).

- **Lectura en voz alta (TTS):** lee las pistas de Sócrates, el resultado de la ejecución, el enunciado del reto y el propio código línea por línea, indicando la sangría. La velocidad de la voz es regulable.
- **Micrófono continuo y sin eco:** el reconocimiento se reinicia solo cuando el navegador lo corta por silencio y se pausa mientras el sistema habla, para no interpretar su propia voz como una orden.
- **Señales sonoras (earcons) y ayuda contextual:** tonos breves generados con AudioContext confirman o rechazan cada orden. Si una frase no se entiende, el sistema lo dice y sugiere «¿Qué puedo decir?», que lee los comandos disponibles en la pantalla actual.

La Tabla 2 resume los comandos disponibles.

*Tabla 2: Comandos de voz disponibles (no requieren frases exactas).*

| Categoría | Ejemplos de frases | Acción del sistema |
|---|---|---|
| Navegación | «Llévame al laboratorio», «Ir al mapa», «Ver mi mascota», «Panel docente», «Volver» | Cambia de pantalla; «Volver» regresa a la anterior usando un historial. |
| Ejercicios y salas | «Ejercicio tres», «Siguiente ejercicio», «Entrar a la sala dos» | Abre el reto indicado y lee su enunciado, o se conecta a la sala N. |
| Laboratorio | «Ejecutar código», «Corre el programa», «Dame una pista», «No entiendo» | Ejecuta el código en el sandbox o solicita una pista socrática. |
| Lectura en voz alta | «Lee mi código», «Lee el resultado», «Lee el enunciado», «¿Qué dijo Sócrates?», «Repite» | Lee el código línea por línea, la consola, el reto o la última pista. |
| Dictado de Python | «Escribe variable edad igual a 15»<br>«Escribe si edad mayor que 18»<br>«Escribe para i en rango de 10»<br>«Escribe definir función sumar con parámetros a y b»<br>«Modo dictado» | `edad = 15`<br>`if edad > 18:`<br>`for i in range(10):`<br>`def sumar(a, b):`<br>Todo lo dicho se escribe como código. |
| Edición | «Deshacer», «Borrar última línea», «Borrar todo», «Salir del bloque» | Revierte cambios, elimina líneas o reduce la sangría de la siguiente línea. |
| Accesibilidad | «Más rápido», «Más lento», «Alto contraste», «Letra más grande», «Silencio», «Apaga el micrófono» | Ajusta la voz y la presentación visual, o detiene la escucha. |
| Ayuda | «¿Qué puedo decir?», «¿Dónde estoy?» | Lee los comandos de la pantalla actual o anuncia la pantalla. |

La Figura 3 muestra el micrófono en estado «Escuchando...» y, en la consola del navegador, la frase reconocida. La Figura 4 muestra el resultado de dictar tres instrucciones: la sangría del `print` se generó automáticamente.

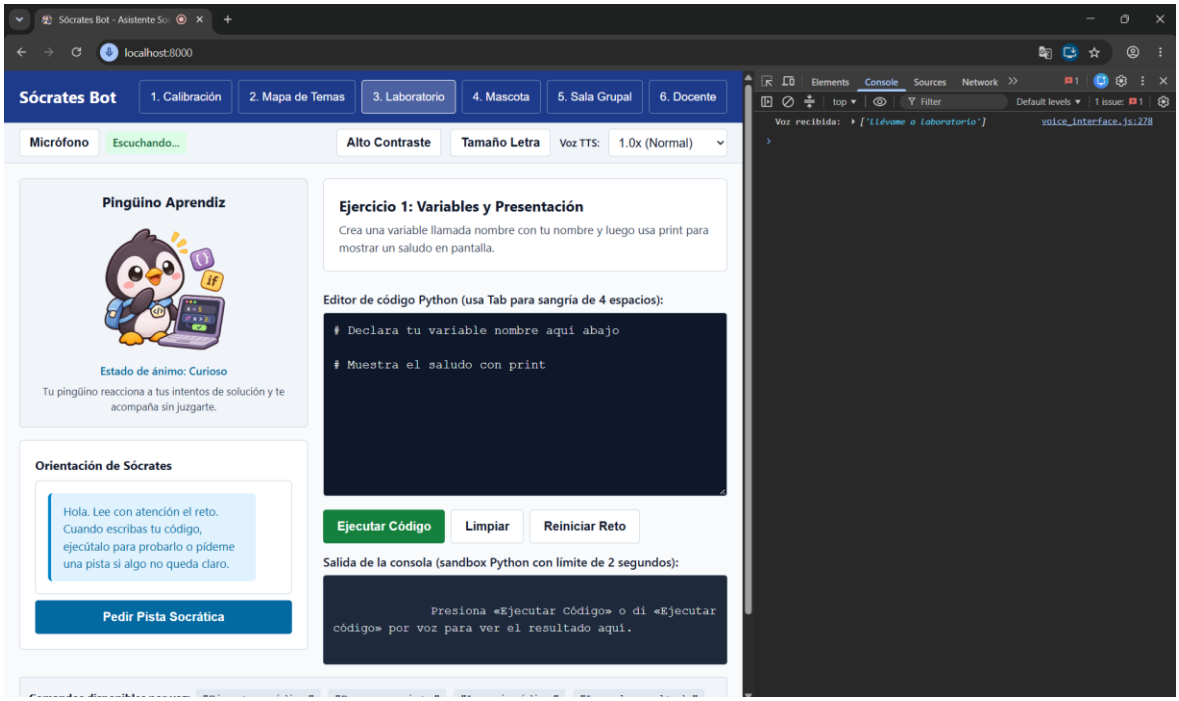



*Figura 3: Comando «Llévame al laboratorio» reconocido; el indicador muestra «Escuchando...».*

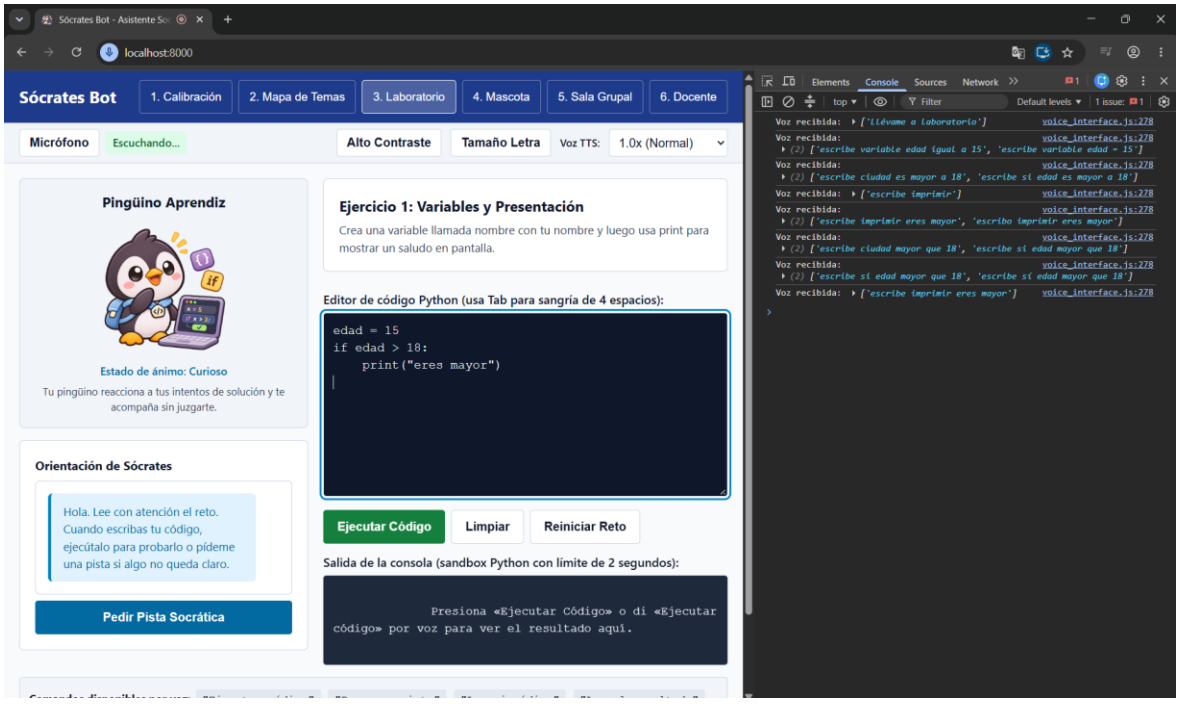



*Figura 4: Código dictado por voz con sangría automática y alternativas del reconocedor en la consola.*

## 3.2. Capa de abstracción para modelos de lenguaje (Groq API y respaldo)

Para la orientación socrática se construyó una capa desacoplada que se comunica con la API de Groq (con la clave obtenida en console.groq.com/keys) y que cambia automáticamente a un modo local cuando no hay conexión:

- **Configuración simple por archivo:** se creó el archivo `.env.example` y la lectura desde `.env`, para que cada estudiante o evaluador coloque su clave sin modificar el código.
- **Conector remoto de Groq:** usa el modelo Llama 3.3 70B, que ofrece respuestas de baja latencia. El modelo recibe una instrucción estricta: nunca entregar el código completo ni resolver el reto, sino formular preguntas breves en español correcto y fáciles de narrar por voz.
- **Cambio automático a modo local (failover):** si la red se cae, la clave se agota o la solicitud tarda demasiado, el sistema conmuta a un motor socrático local que analiza los errores más frecuentes (sangría, dos puntos faltantes, variables inexistentes, paréntesis sin cerrar) y formula la pregunta adecuada sin detener la sesión.

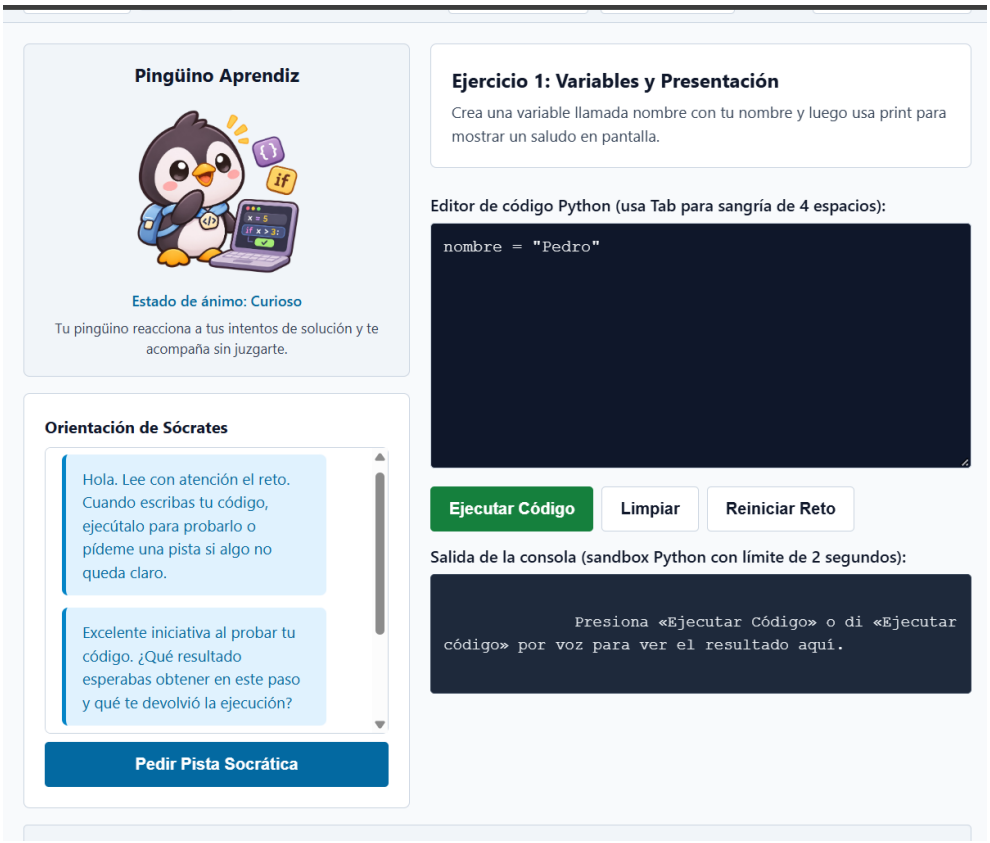



*Figura 5: Pista socrática en forma de pregunta, sin revelar la solución.*

## 3.3. Plataforma PWA (laptop y celular)

La aplicación se construyó bajo el estándar de Aplicación Web Progresiva (PWA):

- **Adaptable a cualquier pantalla:** estructura flexible y botones de al menos 44 × 44 píxeles, cómodos tanto en una pantalla de computadora como en un teléfono (Figura 6).
- **Instalable y tolerante a fallas de red:** cuenta con `manifest.json` y un service worker (`sw.js`) que guarda en caché los archivos principales para que la aplicación abra aunque la conexión sea inestable.
- **Alto contraste y tamaño de letra:** ambos se activan con un botón o por voz (Figura 7).

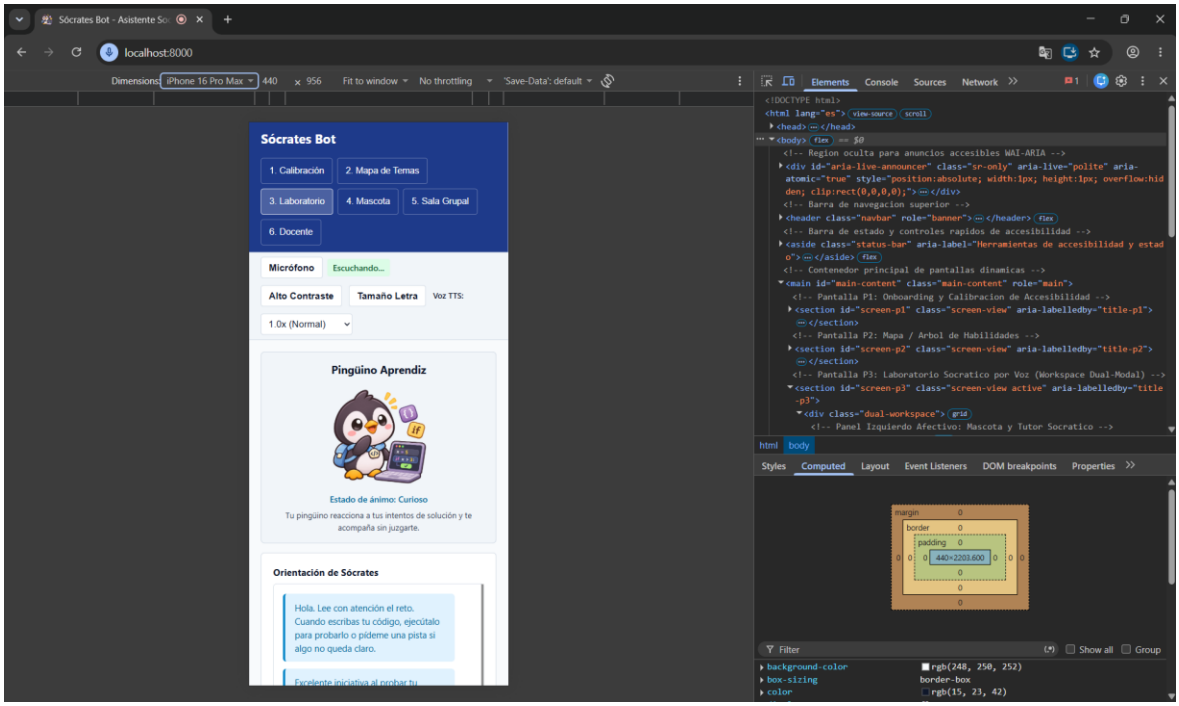



*Figura 6: Vista del laboratorio en un celular de 440 px de ancho (iPhone 16 Pro Max).*

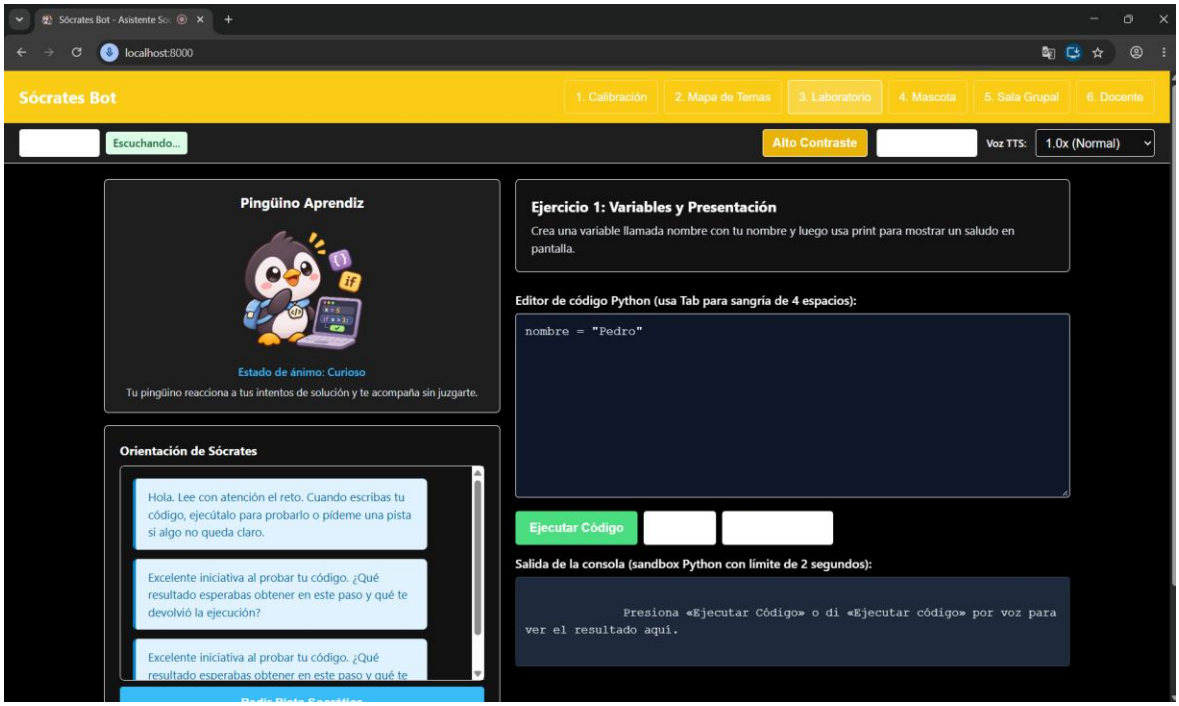



*Figura 7: Modo de alto contraste activado en el laboratorio.*

## 3.4. Personalización continua del estudiante

El sistema no trata a todos los estudiantes por igual, sino que aprende de ellos conforme practican:

- **Calibración inicial:** el estudiante elige su perfil de interacción (estándar, discapacidad motora o discapacidad visual) y consulta la guía de comandos de voz (Figura 8).
- **Perfil persistente:** guarda las preferencias de accesibilidad (alto contraste, velocidad de voz, tamaño de letra) para no tener que configurarlas cada vez.
- **Ritmo y dificultades:** registra los intentos, los aciertos y los temas con más errores (variables, condicionales, bucles o funciones).
- **Detección del estado de ánimo:** si el estudiante falla tres veces seguidas, el sistema infiere que podría estar frustrado: suaviza la pista y el pingüino adopta una actitud solidaria que invita a hacer una pausa.

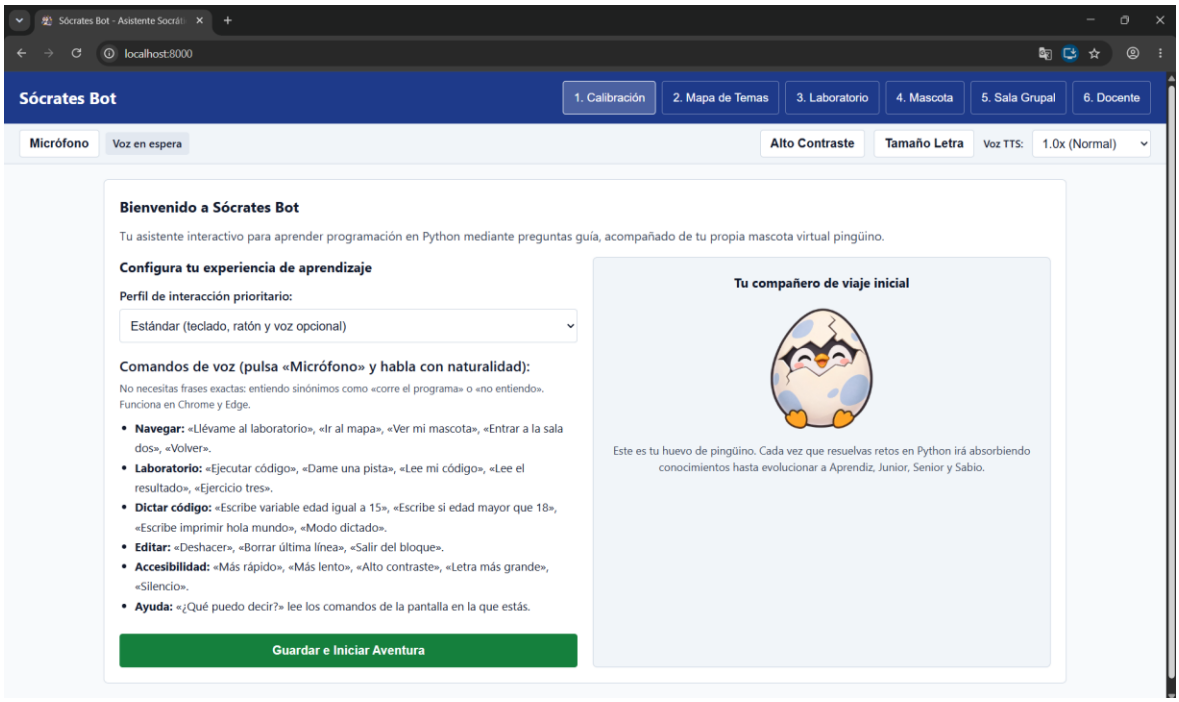



*Figura 8: Pantalla de calibración con el perfil de interacción y la guía de comandos de voz.*

## 3.5. Trabajo multiusuario y panel docente

Para promover el aprendizaje grupal y el seguimiento docente:

- **Salas en vivo:** mediante WebSockets, los alumnos entran a una sala común y escriben en una pizarra de código compartida que se actualiza al instante en todas las ventanas conectadas (Figura 9).
- **Acompañamiento de mascotas:** la sala muestra a los compañeros conectados y la fase de su pingüino.
- **Panel docente:** presenta métricas del aula y el estado de cada estudiante (avanzando, en riesgo o destacado) para detectar bloqueos a tiempo (Figura 10).

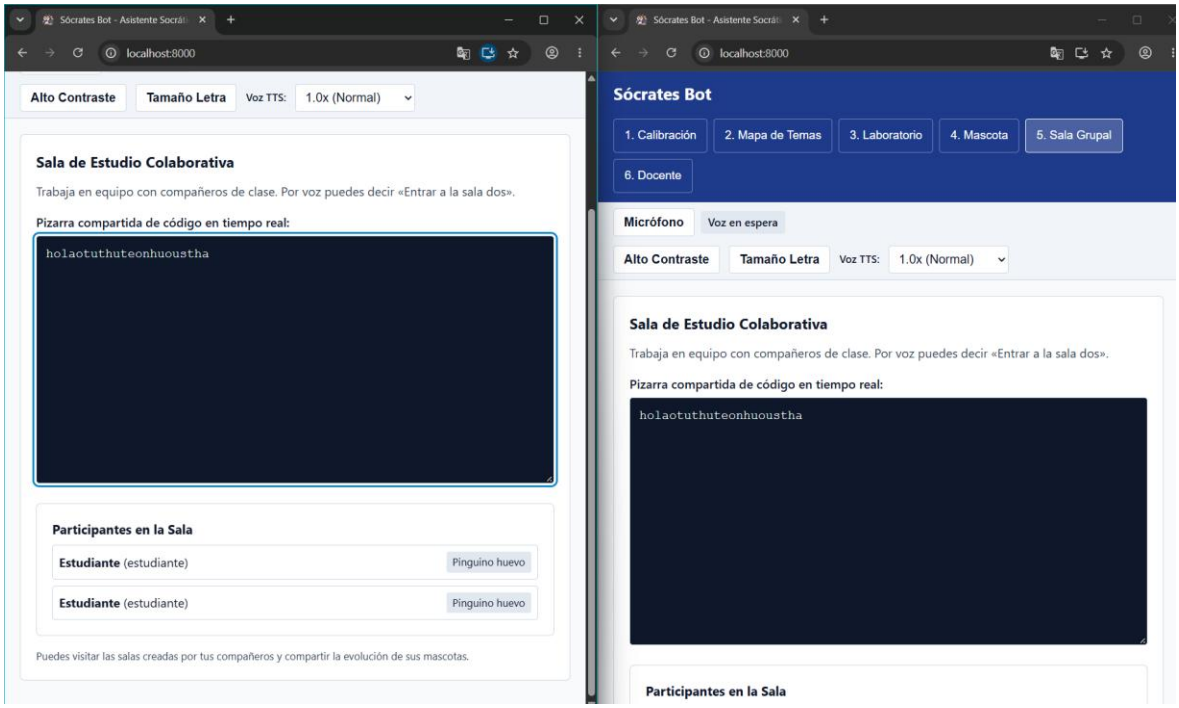



*Figura 9: Dos ventanas en la misma sala: el texto escrito en una aparece en la otra.*

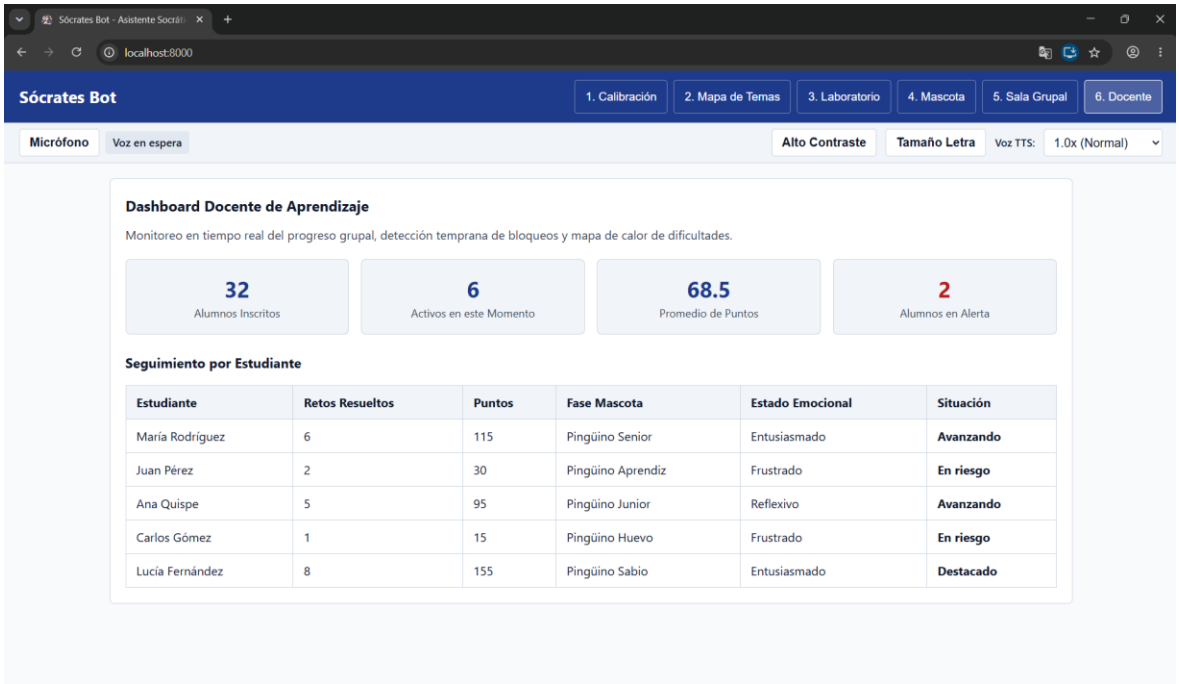



*Figura 10: Panel docente con métricas del aula y seguimiento por estudiante.*

## 3.6. Diseño emocional de Norman y la mascota pingüino

Aplicando la teoría de diseño emocional de Donald Norman (2004), la mascota no es un adorno, sino un puente motivacional estructurado en tres niveles:

- **Nivel visceral (lo que se siente al ver):** expresiones amigables y colores claros que generan agrado a primera vista y reducen la ansiedad frente al código.
- **Nivel conductual (cómo responde durante el uso):** cada vez que el alumno ejecuta su código, el pingüino reacciona: festeja si funciona y muestra apoyo si hay errores persistentes (Figura 12).
- **Nivel reflexivo (la huella a largo plazo):** a medida que el estudiante asimila conceptos, el pingüino pasa por 5 etapas de crecimiento que le recuerdan lo aprendido (Figura 11):
- **1. Huevo (0 a 19 puntos):** representa el inicio del viaje y la curiosidad.
- **2. Aprendiz (20 a 49 puntos):** rompe el cascarón cuando el alumno domina variables y condicionales.
- **3. Junior (50 a 89 puntos):** usa audífonos de programador cuando el alumno domina bucles `for` y listas.
- **4. Senior (90 a 139 puntos):** usa lentes de lectura al dominar la modularización mediante funciones.
- **5. Sabio (140 puntos o más):** lleva birrete y bastón; ha alcanzado la madurez para ayudar a otros en las salas.

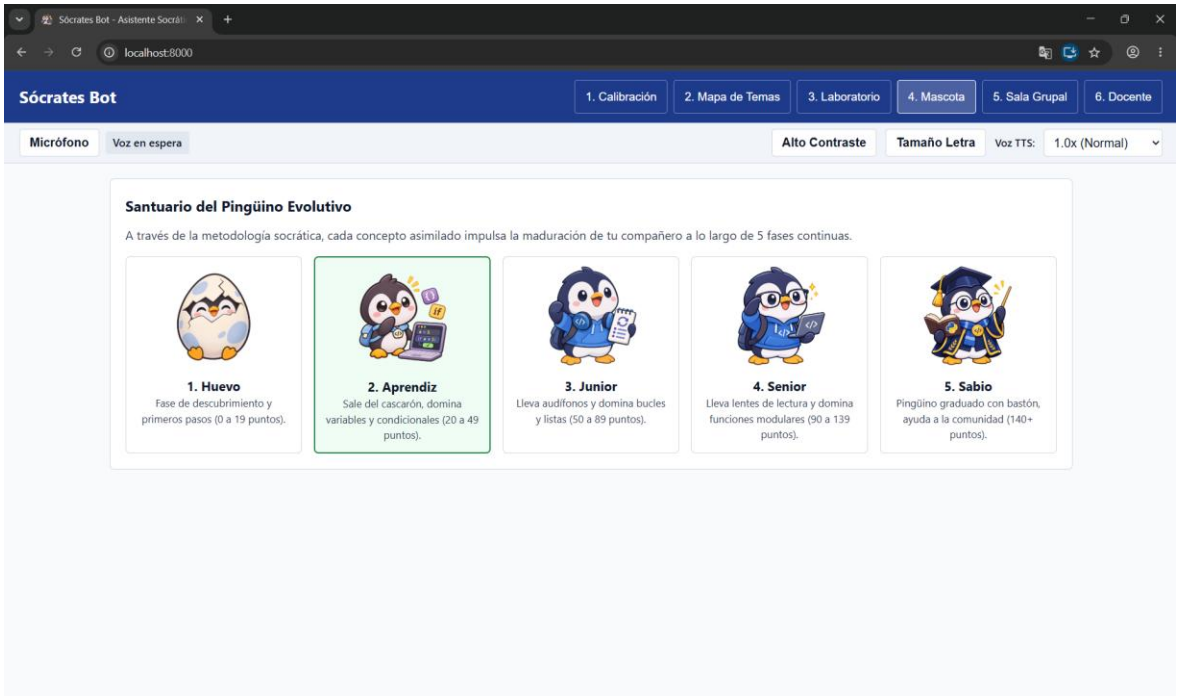



*Figura 11: Santuario con las 5 fases evolutivas del pingüino (fase actual resaltada).*

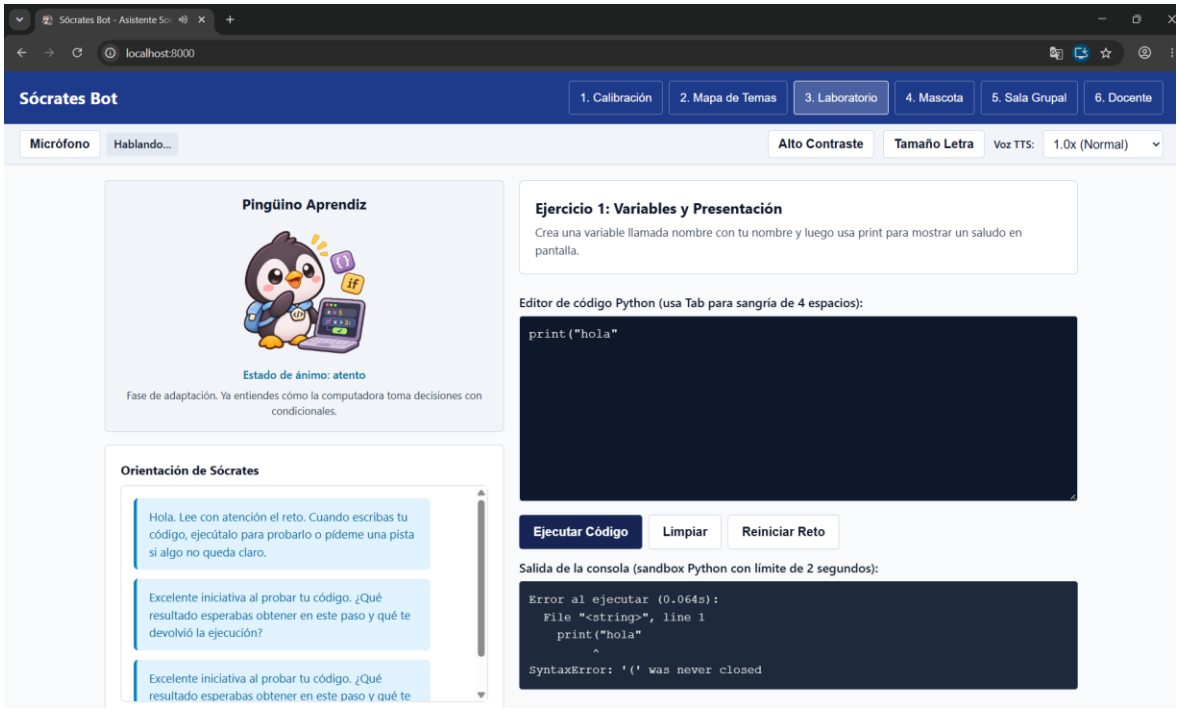



*Figura 12: Error de sintaxis en la consola y reacción conductual del pingüino (estado «atento»).*

# 4. Matriz de trazabilidad y cobertura de historias de usuario (BDD)

La Tabla 3 relaciona cada requerimiento con su historia de usuario, su criterio de aceptación en formato BDD, el módulo de código que lo implementa y la evidencia que lo verifica.

*Tabla 3: Matriz de trazabilidad de requerimientos, historias de usuario y evidencias.*

| ID req. | Historia de usuario | Criterio de aceptación (BDD) | Módulo de código | Estado y evidencia |
|---|---|---|---|---|
| RF-01 / RNF-03 | HU-01 y HU-03: tutoría socrática sin spoilers y conmutación | Dado un error en el código, cuando el alumno pide ayuda, el tutor responde con una pregunta guía usando Groq o el respaldo local. | `app/socratic_tutor.py` | Verificado en modo local (Fig. 5) |
| RF-02 / RNF-01 | HU-01: control total por voz y dictado en español | Dado un usuario con movilidad reducida, cuando dice «Ejecutar código» o dicta instrucciones, el sistema ejecuta y escribe sin usar el ratón. | `static/js/voice_interface.js` | Verificado (Fig. 3 y 4) |
| RF-03 / RNF-01 | HU-02: accesibilidad auditiva y lectores de pantalla | Dado un alumno con ceguera, cuando cambia el estado de la mascota o llega una pista, se actualiza `aria-live` y se lee por TTS. | `templates/index.html` `static/js/app.js` | Verificado; alto contraste (Fig. 7) |

| ID req. | Historia de usuario | Criterio de aceptación (BDD) | Módulo de código | Estado y evidencia |
|---|---|---|---|---|
| RF-05 / RNF-05 | HU-03: ejecución segura en sandbox de Python | Dado un script con bucle infinito o instrucción peligrosa, cuando se ejecuta, se corta en 2.0 s sin congelar el servidor. | `app/code_sandbox.py` | Verificado: corte a 2.007 s (Fig. 13) |
| RF-06 / RF-08 | HU-02 y HU-05: mascota evolutiva y personalización continua | Dado que el alumno acumula puntos, cuando sube de nivel, el pingüino cambia de fase y el perfil se actualiza. | `app/pet_manager.py` y `app/user_profile.py` | Verificado (Fig. 11 y 12) |
| RF-07 / RF-04 | HU-04 y HU-06: salas colaborativas y panel docente | Dados dos alumnos en la misma sala, cuando uno escribe, el código se sincroniza por WebSocket y el docente visualiza alertas. | `app/rooms_manager.py` `main.py` | Verificado (Fig. 9 y 10) |

# 5. Evidencia de pruebas funcionales

Las pruebas se realizaron en Google Chrome sobre la aplicación ejecutada localmente. Además, el intérprete de comandos de voz se probó de forma automatizada con 22 frases de comando y 21 frases de dictado, y la sincronización de salas con dos clientes WebSocket simultáneos. La Tabla 4 resume los casos.

*Tabla 4: Casos de prueba funcionales ejecutados.*

| ID | Caso de prueba | Resultado esperado | Resultado obtenido | Evid. |
|---|---|---|---|---|
| PF-01 | Decir «Llévame al laboratorio» | Navega a P3 y lo confirma por voz | Aprobado | Fig. 3 |
| PF-02 | Decir «Ejecutar código» (con tilde) | Ejecuta el código del editor | Aprobado | Automatizada |
| PF-03 | Dictar variable, condicional y `print` | Código válido con sangría automática | Aprobado | Fig. 4 |
| PF-04 | El reconocedor entrega «ciudad…» y «si edad…» | Se elige la alternativa que forma un `if` | Aprobado | Fig. 4 |
| PF-05 | Pedir una pista con error en el código | Pregunta guía sin revelar la solución | Aprobado (modo local) | Fig. 5 |
| PF-06 | Ejecutar `print("hola"` | Error legible en consola y reacción del pingüino | Aprobado | Fig. 12 |
| PF-07 | Ejecutar `while True: pass` | Corte a los 2.0 s sin bloquear el servidor | Aprobado (2.007 s) | Fig. 13 |
| PF-08 | Escribir en la pizarra con dos ventanas en la sala | El texto aparece en la otra ventana | Aprobado | Fig. 9 |

| ID | Caso de prueba | Resultado esperado | Resultado obtenido | Evid. |
|---|---|---|---|---|
| PF-09 | Abrir la aplicación con 440 px de ancho | Navegación y contenido se reacomodan | Aprobado | Fig. 6 |
| PF-10 | Activar alto contraste | Colores de alto contraste en toda la interfaz | Aprobado | Fig. 7 |
| PF-11 | Abrir el panel docente | Métricas y estado legible por estudiante | Aprobado | Fig. 10 |

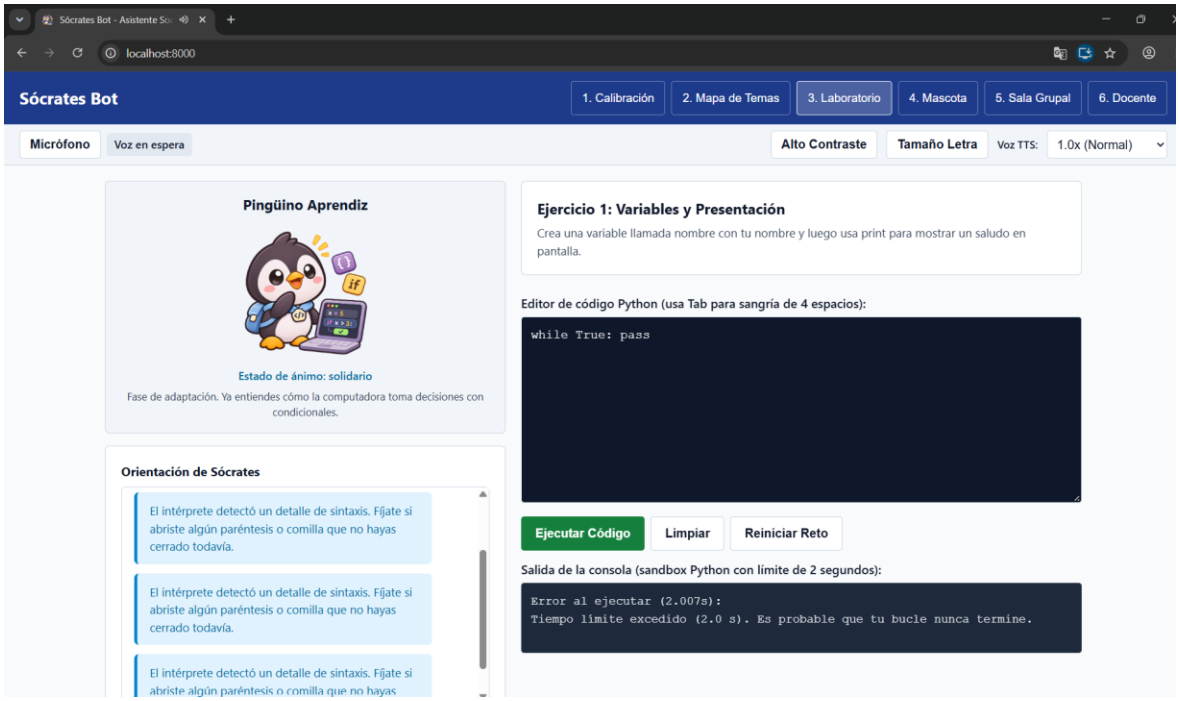



*Figura 13: Bucle infinito detenido por el sandbox a los 2.007 segundos.*

## 5.1. Hallazgos corregidos durante las pruebas

Las pruebas permitieron detectar y corregir problemas reales de interacción:

- **Tildes en la transcripción:** el navegador transcribe «código» con tilde, pero los comandos se comparaban sin ella, por lo que órdenes como «Ejecutar código» nunca se reconocían. Se agregó la normalización del texto.
- **Palabras parecidas:** «si edad» se reconocía a veces como «ciudad». Se pasó a evaluar todas las alternativas del reconocedor y elegir la que forma una instrucción válida.
- **Eco del sintetizador:** el micrófono captaba la voz del propio sistema. Ahora se pausa mientras el sistema habla.
- **Salas colaborativas:** el servidor rechazaba las conexiones por falta de la librería de WebSockets. Se cambió la dependencia a `uvicorn[standard]`.

### 5.2. Limitaciones conocidas

- El reconocimiento de voz funciona en Chrome y Edge; navegadores como Brave o Firefox no lo soportan, y requiere conexión a internet.
- Mientras el sistema habla, el micrófono está en pausa, por lo que una lectura larga no puede interrumpirse por voz (sí con el botón del micrófono).

## 6. Módulos de código desarrollados al 70%

Los siguientes archivos del repositorio sustentan el avance funcional:

- **config.py:** lectura centralizada de `GROQ_API_KEY` desde `.env`, selección del modelo Llama 3.3 y límites de tiempo del sandbox.
- **app/socratic_tutor.py:** capa desacoplada de inteligencia artificial con patrón de estrategia: conexión HTTP con Groq y motor de respaldo con reglas pedagógicas.
- **app/code_sandbox.py:** ejecución aislada de Python en subprocesos con límite estricto de 2.0 segundos y lista de operaciones bloqueadas.
- **app/pet_manager.py:** modelo del pingüino: puntos requeridos para cada una de las 5 fases y reacciones emocionales según Norman.
- **app/user_profile.py:** persistencia en JSON del estado del estudiante: ritmo, puntos, errores por tema e inferencia de frustración.
- **app/rooms_manager.py:** coordinación de salas grupales en tiempo real mediante WebSockets.
- **main.py:** servidor FastAPI con los endpoints de retos, ejecución, pistas socráticas, analítica docente y WebSockets.
- **templates/index.html:** estructura semántica accesible con las 6 pantallas (P1 Calibración, P2 Mapa, P3 Laboratorio, P4 Santuario, P5 Sala, P6 Docente) y roles ARIA.
- **static/css/styles.css:** estilos con soporte de alto contraste, letra grande, botones táctiles de 44 px y diseño responsive para celular y laptop.
- **static/js/voice_interface.js:** controlador del micrófono (Web Speech API), normalización de texto, intérprete de intenciones con sinónimos, dictado de Python con sangría automática, earcons y síntesis de voz.
- **static/js/app.js:** navegación entre pantallas con historial, conexión con el backend y actualización de la mascota, la sala y el panel docente.
- **static/img/mascota/:** imágenes de las 5 fases evolutivas del pingüino.
- **data/simulated_data.json:** banco de 8 retos de Python ordenados por dificultad y datos simulados de un aula de 32 estudiantes (métricas generales y detalle de 5 estudiantes) para el panel docente.

# 7. Delimitación del alcance

*Tabla 5: Delimitación del alcance entre el avance actual y la entrega final.*

| Área del proyecto | Implementado y funcional al 70% | Postergado para la entrega final (100%) |
|---|---|---|
| Inteligencia socrática | Conector activo con Groq y conmutación a reglas locales ante pérdida de red. | Soporte de modelos locales en hardware dedicado mediante Ollama. |
| Ejecución de código | Sandbox por subproceso independiente con límite de 2.0 s y bloqueo de comandos destructivos. | Aislamiento en contenedores Docker efímeros por sesión de usuario. |
| Mascota virtual | Evolución gráfica y conceptual en 5 fases con diseño emocional de Norman. | Animaciones vectoriales e interacción táctil en tiempo real. |
| Interfaz y accesibilidad | Navegación manos libres por voz en lenguaje natural, dictado de Python, síntesis TTS, compatibilidad con lectores de pantalla y alto contraste. | Evaluación formal con usuarios reales con ceguera y movilidad reducida, con reporte SUS. |
| Analítica docente | Panel operativo con datos simulados, estados por estudiante y mapa de errores por tema. | Algoritmo predictivo de abandono y exportación automática de informes en PDF. |

# 8. Cronograma y próximos pasos

La Figura 14 muestra el cronograma del proyecto planificado en la PC02. Las fases 1 y 2 están concluidas y la fase 3 (integración de IA, voz y personalización) se encuentra operativa en este avance. Para la entrega final quedan:

- **Pruebas con usuarios (fase 4.2):** evaluación con personas con discapacidad visual y motora, aplicando el cuestionario SUS y verificando los criterios de WCAG 2.1 nivel AA.
- **Aislamiento del sandbox:** migración de la ejecución a contenedores Docker efímeros por sesión.
- **Modelos locales:** integración de Ollama como proveedor de respaldo en lugar de las reglas locales.
- **Analítica avanzada:** predicción de abandono y exportación de informes del aula en PDF.
- **Despliegue final (fase 4.3):** publicación de la PWA y sustentación del proyecto.

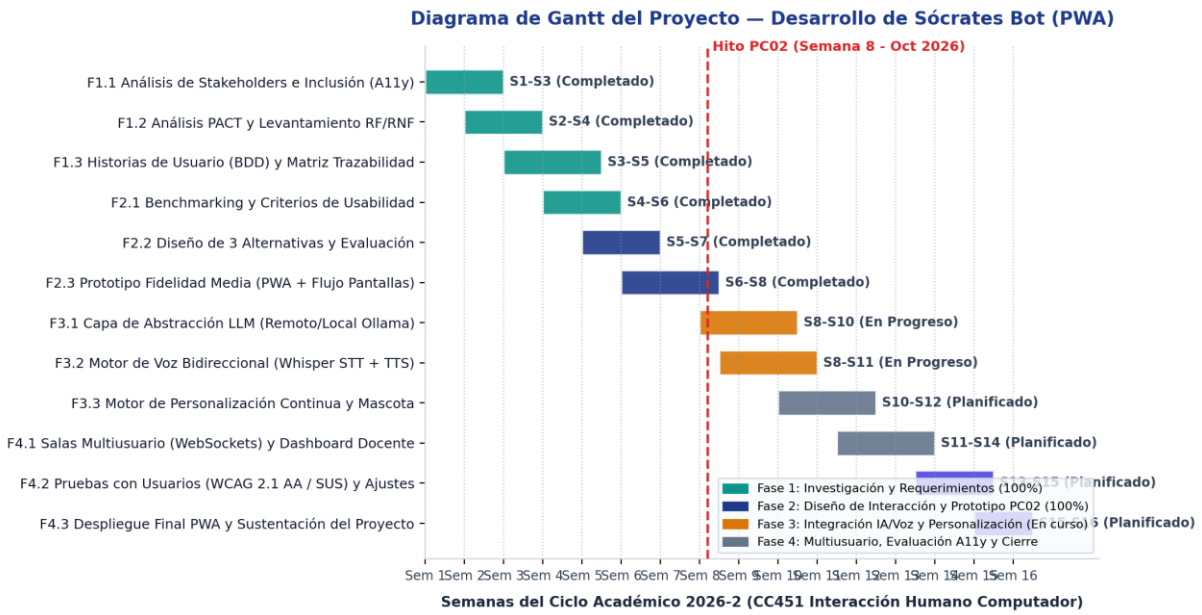



*Figura 14: Diagrama de Gantt del proyecto (planificación de la PC02).*

# 9. Conclusiones

- Se alcanzó el 70% del proyecto con las seis pantallas operativas y cada requerimiento obligatorio de la PC03 verificado con evidencia visual o pruebas automatizadas.
- La interfaz de voz evolucionó de comandos rígidos a lenguaje natural. La principal lección de interacción fue que el reconocedor produce variantes (tildes, palabras parecidas como «si edad» y «ciudad»), y que el diseño debe tolerarlas en lugar de exigir frases exactas al usuario.
- La retroalimentación multimodal (síntesis de voz, earcons y anuncios `aria-live`) permite que cada acción sea perceptible sin necesidad de ver la pantalla, lo que es clave para los perfiles de discapacidad visual y motora.
- La capa de abstracción con respaldo local garantiza que la tutoría socrática continúe aun sin conexión o sin clave de API, sin interrumpir la sesión del estudiante.
- El siguiente paso crítico es la evaluación con usuarios reales, que validará con métricas (SUS y WCAG 2.1 AA) las decisiones de diseño tomadas hasta ahora.